

From Text to Intelligence: Building Knowledge Graphs with LLMs

July 2026

Contents

From Text to Intelligence: Building Knowledge Graphs with LLMs	1
0. The Big Picture	1
1. Foundations: Why Ontologies, Why Knowledge Graphs	2
2. Step 1 — The Ontology in Code (code/ontology.py)	7
3. Step 2 — LLM Ingestion: Text → Triples (code/extract_llm.py)	8
4. Step 3 — Building the Graph (code/build_graph.py)	10
5. Step 4 — Querying the Graph (code/query_graph.py)	11
6. Step 5 — Machine Learning on the Graph: Predicting Missing Links	12
7. Step 6 — The Natural-Language Interface (code/nl_query.py)	15
8. Putting It Together: What Runs on a Knowledge Graph	16
9. Exercises	17
10. Reading list	17
Appendix: File map & how to run	17

From Text to Intelligence: Building Knowledge Graphs with LLMs

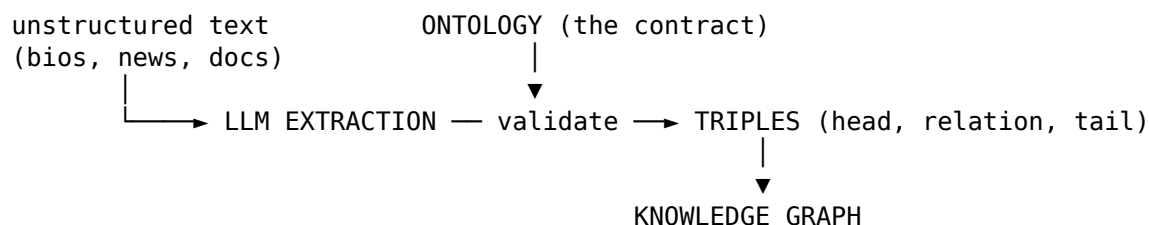
A complete, hands-on tutorial: design an ontology, use an LLM to extract entities and relationships from raw text, build a knowledge graph, query it, run machine learning on it to predict missing links (LinkedIn-style), and put a natural-language interface on top.

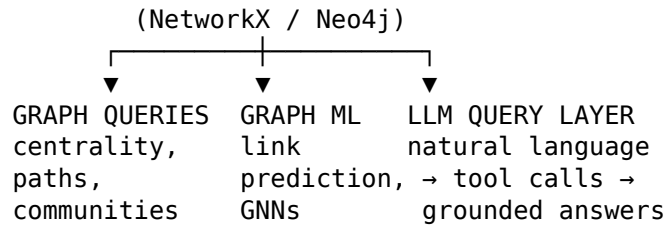
Every code file referenced here lives in code/ and actually runs — the outputs shown are real. The only steps needing an Anthropic API key are the two LLM ones (extraction and NL querying); a pre-extracted dataset is included so everything else runs offline.

```
pip install networkx scipy anthropic torch torch_geometric
```

0. The Big Picture

Everything in this tutorial is one pipeline:





Two ideas make this architecture powerful, and they are mirror images of each other:

1. **LLM on the way in.** LLMs are the best general-purpose information extractors ever built. Given a schema, they turn messy prose into clean structured facts.
2. **LLM on the way out.** LLMs are unreliable *stores* of facts (they hallucinate) but excellent *translators* of intent. So we never ask the LLM what it knows — we ask it to translate a user’s question into graph operations, and the graph answers.

The graph in the middle is the system of record: every fact is typed, validated against the ontology, and traceable to a source document.

We build a miniature LinkedIn: people, companies, skills, schools, and locations, extracted from ten profile bios and news snippets. Small enough to inspect by hand, rich enough to demonstrate “who is the most connected person?”, “how do I reach X?”, “people you may know”, and “this Google ML engineer probably knows TensorFlow even though they never listed it” — the exact inference LinkedIn describes doing with graph neural networks on its member knowledge graph.

1. Foundations: Why Ontologies, Why Knowledge Graphs

This chapter is the “why” behind everything else in the tutorial. If you understand it deeply, every later step will feel inevitable rather than arbitrary.

1.1 The problem all of this solves: strings, not things

Start from the failure mode. In an ordinary database or document store, the token “Google” appears in an employer column here, a customer field there, a free-text bio somewhere else. Nothing in the system knows these are the *same real-world entity*, that this entity is a *company*, that companies *employ people* and *use technologies*, or that “Google LLC”, “google”, and “Alphabet’s search division” might refer to it too. The meaning lives in the heads of developers and in application code — not in the data. Every new question (“who works at companies that use Kubernetes?”) requires a developer to rediscover that meaning and encode it as a bespoke join.

When Google launched its Knowledge Graph in 2012, its slogan named this exact gap: “**things, not strings.**” A knowledge graph replaces string co-occurrence with an explicit model of the world: entities with identity and type, connected by typed, directional relationships. The ontology is the part of that model that says *what kinds of things and relationships are allowed to exist* — the world’s grammar, separate from the world’s facts.

Precisely, three layers:

- A **triple** is an atomic fact: (head, relation, tail) — (Amina Yusuf, WORKS_AT, Google). Everything a KG asserts decomposes into triples plus attributes.
- An **ontology** defines the vocabulary of types those facts must use: entity types (Person, Company, Skill...), relation types (WORKS_AT, KNOWS...), what each relation may connect (**domain** = allowed head type, **range** = allowed tail type), plus attributes and rules (KNOWS is symmetric; WORKS_AT means *current* employment, distinct from PREVIOUSLY_AT). In knowledge-representation jargon this is the **TBox** (terminology).
- The **knowledge graph** is the ontology *instantiated* with actual entities and relationships — the **ABox** (assertions). Nodes = entities, edges = relations, both carrying properties.

A database schema says what’s *storable*; an ontology also says what’s *meaningful* and *inferable*. That distinction sounds philosophical until Section 1.4, where each word turns into a concrete engineering capability.

Two subtleties you should carry with you. **Class vs instance**: Person is a class; Amina Yusuf is an instance. But is Python a class of things or a single thing? For our purposes it is an *instance* of the class Skill — one canonical node everyone links to — and that choice (rather than, say, making each person’s Python proficiency its own node) is exactly what makes skills traversable and countable. Getting class/instance boundaries wrong is the most common beginner modeling error. **Identity**: every entity needs one canonical identifier so that all facts about it accumulate on one node. Formal systems use IRIs (<https://example.org/person/amina-yusuf>); our small graph uses canonical names; production systems use opaque IDs plus alias tables. The moment identity fragments (“Google” ≠ “Google LLC”), the graph silently stops being a graph of *things* and reverts to strings.

One more piece of background that trips people up later: the **open-world assumption**. A KG (unlike a typical SQL application) does not treat absence as falsehood — if there is no HAS_SKILL edge from Ben to TensorFlow, that means *unknown*, not *no*. This worldview is why “KG completion” (Section 6) is a legitimate ML task rather than nonsense: we treat the graph as a partial observation of reality and predict what’s missing.

1.2 A short history, so the vocabulary makes sense

The ideas are old; the scale and the LLMs are new. **Semantic networks** (1960s, Quillian) first proposed representing knowledge as concept-nodes and labeled links. **Frames** (Minsky, 1970s) added structured slots — recognizably today’s “entity with typed attributes.” **Description logics** (1980s-90s) made class/relation definitions mathematically precise so machines could *reason* over them — compute class membership, detect contradictions — and became the formal core of **OWL**. The **Semantic Web** program (Berners-Lee, 2001) standardized the stack — RDF for triples, OWL for ontologies, SPARQL for queries — aiming at a machine-readable web. Its grand vision stalled (too much formality, too little incentive), but the tooling matured. Then industry quietly picked up the useful parts at enormous scale: Google’s Knowledge Graph (2012, search), Facebook’s entity graph, Amazon’s product graph, Microsoft’s academic and enterprise graphs, and LinkedIn’s **Economic Graph** — members, organizations, skills, titles, schools, geos, jobs. The industrial systems dropped heavyweight logical reasoning in favor of pragmatic property graphs plus statistics and, later, graph ML. Since ~2023, LLMs removed the biggest historical bottleneck — getting knowledge *into* the graph — which is why this architecture is having a renaissance now.

Hence **two modeling traditions**, both alive:

	RDF / OWL (semantic web)	Labeled Property Graph (LPG)
Fact unit	triple (subject-predicate-object)	node/edge with properties
Schema	OWL ontologies, formal logic, reasoners	looser; constraints per-database
Query	SPARQL	Cypher / Gremlin
Tools	rdflib, Protégé, GraphDB, Jena	Neo4j, Neptune, Memgraph, NetworkX
Strength	interoperability, formal inference	developer ergonomics, analytics, ML

Industry KGs are overwhelmingly property-graph-shaped in practice, so this tutorial uses the LPG style — plain Python classes for the ontology, NetworkX for the graph, Cypher equivalents

shown throughout. The concepts transfer 1:1 to RDF/OWL if your domain demands formal reasoning or cross-organization data exchange (medicine and life sciences often do: SNOMED, Gene Ontology).

1.3 Why an ontology is *needed* — six concrete returns

Each of these is a capability you lose without one.

1. A shared contract between humans, systems, and now LLMs. An ontology is an API for facts. Two teams (or two pipelines, or you-today and you-in-six-months) can independently produce and consume graph data and still interoperate, because WORKS_AT means one agreed thing. In the LLM era this contract has a new, decisive job: it is *the specification you hand the model*. Ask an LLM to “extract entities and relationships” with no ontology and you get schema drift — the same fact comes back as WORKS_AT, EMPLOYED_BY, worksFor, or is employed at on different calls; entity types proliferate (Organization, Company, Employer, Tech Giant); and after a thousand documents your “graph” is an unqueryable soup of near-synonyms. **Without an ontology, an LLM-built knowledge graph is write-only.** This failure mode is the single most common reason LLM-KG projects die, and it is fully preventable by doing what Step 2 of this tutorial does: put the closed vocabulary in the prompt and validate against it on the way out.

2. A validation gatekeeper. Because the ontology is machine-readable, it mechanically rejects malformed facts — unknown relations, domain/range violations (Company WORKS_AT Person), type confusions. LLMs are probabilistic; your graph’s integrity shouldn’t be. The pattern from Section 3 — *the LLM proposes, the ontology disposes* — is the hallucination firewall, and it only exists if the ontology does.

3. Query power. Every query in Section 5 leans on types. “People with PyTorch at Google” is only expressible because Person, Skill, Company, HAS_SKILL, and WORKS_AT are distinct, typed constructs. An untyped edge soup can tell you *that* things are connected, never *what the connection means*. The ontology defines the query language of your domain: you can only ask what you modeled.

4. Inference — new facts from old, by rule. Semantics make deduction possible. Because KNOWS is declared *symmetric*, A KNOWS B yields B KNOWS A (our graph builder materializes exactly this). Declare LOCATED_IN *transitive* and office in London + London in UK yields office in UK. Declare MANAGES the *inverse* of REPORTS_TO and each direction implies the other. Add subclass hierarchies (PyTorch IS_A DeepLearningFramework IS_A Skill) and a query for “deep-learning people” finds PyTorch users without anyone asserting it. This rule-based inference is the classical, deterministic sibling of the statistical inference (link prediction) in Section 6 — mature systems run both, and both need declared semantics to run at all.

5. A substrate for machine learning. All of Section 6’s power comes from *heterogeneity with meaning*: metapaths like Person→Company→Skill exist only because types exist; relation embeddings (DistMult’s per-relation vector) exist only because relations are a closed set; the GNN’s node features start with the type one-hot. The ontology is literally the feature space.

6. Evolvability and governance. A KG outlives any one application. The ontology, versioned as code and reviewed like an API, is what lets it evolve deliberately — you can see that adding Job postings is additive (safe) while redefining WORKS_AT semantics is breaking (requires re-extraction and migration). Without it, every change is archaeology.

1.4 Why a *graph* — and, honestly, when not

Given typed facts, why store them as a graph rather than tables or documents? Because of what the workload is. Professional-network questions are **relationship-first and multi-hop**: paths, neighborhoods, mutual connections, “friends-of-friends-minus-friends.” In SQL each hop is a join, a 6-hop path query is a 6-way self-join, and variable-length paths (“any path up to length 6”) are painful; in a graph store each hop is a pointer traversal and paths are native primitives. Second,

schema evolution is additive: a new relation type is just new edges — no migrations, no NULL-ridden wide tables — which matters enormously when an LLM keeps discovering new kinds of facts in your documents. Third, **heterogeneous data unifies naturally:** people, skills, jobs, and companies in one structure, connected, rather than normalized into a dozen tables whose connective meaning lives in foreign-key conventions.

And the honest converse — a KG is the *wrong* tool when: your queries are aggregations over homogeneous records (revenue by quarter → SQL/OLAP); you need full-text relevance ranking (→ search index); or your data is genuinely tabular and relationship-free. Vector databases deserve a special mention because they’re the default RAG answer: embeddings retrieve by *similarity*, graphs by *structure*. “Documents like this one” is a vector question; “skills of people who work with Amina” is a graph question — similarity search cannot compose two hops. Production systems increasingly run both, with the KG providing the multi-hop, explainable backbone (see GraphRAG, Section 7).

1.5 How to build an ontology, step by step

Ontology building has a real methodology (academia calls it *ontology engineering* — Ontology 101, METHONTOLOGY, NeOn). Here is the distilled, practical loop, illustrated with how our professional ontology came to be. It is iterative — expect three or four passes, and keep every pass small.

Step 1 — Scope, via competency questions. Do not model “the world”; model *answers*. Write down the 5–15 questions the graph must answer — these are your acceptance tests:

1. Who works where, with what skills, educated where?
2. Which node is the most connected / most “busy”?
3. How are two people connected (shortest warm-intro path)?
4. Who should connect next (“people you may know”)?
5. What skills does a person probably have but hasn’t listed?

Everything not needed by a competency question stays out. This is the discipline that prevents the classic death-by-modeling where a team spends months taxonomizing the universe before storing a single fact.

Step 2 — Study the data, and hunt for existing vocabularies. Read 20–50 real documents from your corpus. What entities recur? What facts do sentences actually assert? Simultaneously, check whether someone already modeled this domain: schema.org (people, orgs, generic things), FOAF (social networks), ESCO/ONET (*skills and occupations* — *LinkedIn’s skill taxonomy is a cousin*), SNOMED/UMLS (*medicine*), FIBO (*finance*). *Reusing even just the names and definitions** from a standard buys interoperability and saves design arguments. Our domain is simple enough to model directly, but its shape deliberately echoes schema.org’s Person/Organization.

Step 3 — Choose entity types (the nouns). From the data study, list candidate nouns and promote the recurring, identity-bearing ones to entity types. The critical test for each candidate is the **node-vs-property decision**: *if you will ever traverse through it, count who shares it, or attach facts to it, it must be a node; if it’s just a descriptive value of one entity, it’s a property*. “London” — traverse through it (“who else is in London?”) → node. “Senior” in a title — descriptive value → property on Person. This one decision, applied to Skill, is what makes the entire ML half of this tutorial work: skills-as-nodes create the Person-Skill-Person paths that link prediction feeds on; skills-as-strings would have made those connections invisible. When in doubt early on, prefer property (cheap to promote to a node later; demoting a node type is a migration).

Step 4 — Choose relation types (the verbs). From the same documents, list the asserted facts and name a relation for each recurring kind. For every relation, decide: **domain and range** (WORKS_AT: Person→Company — this is what makes validation possible); **direction** (pick the natural reading; add inverses only if you need them); **algebraic properties** — symmetric? (KNOWS yes, MANAGES no), transitive? (LOCATED_IN yes, KNOWS emphatically no), one-to-many or many-to-many?; and **granularity** — we split WORKS_AT from PREVIOUSLY_AT be-

cause “current vs former employer” appears in our competency questions; a temporal system would instead use one EMPLOYED_AT edge with start/end date properties. Granularity errors are the expensive ones: too coarse (“RELATED_TO”) and queries can’t discriminate; too fine (WORKS_AT_FULL_TIME, WORKS_AT_CONTRACT, ...) and extraction becomes unreliable and queries verbose. Match granularity to the distinctions your questions actually draw.

Step 5 — Write definitions as if the reader were an LLM — because it will be. Every type and relation gets a one-sentence natural-language definition with examples (“Skill: a technology, tool, or competency, e.g. Python, SQL”). Historically these definitions were documentation; in an LLM-extraction pipeline they are *executable* — they go verbatim into the prompt, and their clarity directly determines extraction quality. Ambiguous definition, ambiguous graph.

Step 6 — Paper-test against the competency questions. Before writing code, take five real documents, extract triples *by hand* under the draft ontology, and try to answer each competency question from the resulting mini-graph on paper. This half-hour exercise catches most design flaws: facts the ontology can’t represent (a person with two current employers? an investment relationship?), questions that need a missing hop, types that never get used. Fix, and re-test.

Step 7 — Encode, with validation. Only now write ontology.py (Section 2): types and relations as data, domain/range as code, validate_triple() as the gatekeeper, to_prompt_block() as the LLM rendering. The ontology lives in version control and changes by review, like any API.

Step 8 — Iterate under governance. Run extraction on real data and *read the validator’s rejection log* — it is your ontology’s error signal. Systematic rejections mean the world contains fact types your model lacks; decide deliberately whether to extend (new relation), refine (split a type), or continue excluding them. Additive changes (new types/relations) are cheap. Semantic changes (redefining a relation) are breaking: they invalidate existing edges, prompts, queries, and trained models — treat them like breaking API changes, versioned and migrated. This asymmetry — *extensions cheap, redefinitions expensive* — is the deepest practical fact about ontologies, and it drives the next section’s answer.

Pitfalls, from the field: over-modeling (ontology bloat nobody populates — LinkedIn runs a trillion edges on roughly a dozen entity types; complexity belongs in scale and inference, not schema); class/instance confusion (Section 1.1); stringly-typed properties that should be nodes; premature hierarchies (add IS_A taxonomies when a competency question needs them, not before); and ignoring n-ary facts — a triple can’t natively say “Amina worked at Google *from 2019 to 2024 as SWE*”; the standard fix is **reification**: promote the fact itself to a node (Employment event with edges to person, company, and date/title properties). Reify only relations that genuinely carry their own attributes; reifying everything doubles your graph’s complexity for nothing.

1.6 Who should design it: you, the LLM, or both?

Now the question behind the question. Three workflows exist.

Pure manual (the classical way): domain experts + a knowledge engineer, whiteboards, weeks. Its real and underrated value is that *the act of modeling is the act of understanding the domain*. Deciding whether Skill is a node, whether WORKS_AT splits from PREVIOUSLY_AT, what “knows” even means — these decisions force you to interrogate your data and your questions, and the person who made them can later extend the system coherently, the way a good API designer keeps an API coherent. For a small ontology (5-15 types — which, per the bloat warning, is what you usually want), manual design is a day’s work, not a bottleneck. Its weaknesses: blind spots (you model the data you *imagine*, and real corpora always contain fact types you didn’t), naming inconsistency, and — for genuinely large domains (hundreds of types: medicine, e-commerce catalogs) — it simply doesn’t scale.

Pure LLM (tempting, fast, wrong): feed sample documents to the model, ask for an ontology, take what comes back. The output will look plausible — that’s the danger. Uncurated LLM ontologies systematically over-generate (25 entity types where 6 would do, one relation per verb it noticed), miss the load-bearing structural decisions (an LLM doesn’t know Skill must be a node *because*

your link-prediction roadmap depends on Person-Skill-Person paths — that’s a fact about your goals, not your text), encode no organizational choices (your team’s identity conventions, compliance constraints, existing taxonomies), and — worst — leave no one who *owns* the model when it needs to evolve. Remember the asymmetry from Step 8: the ontology is the most expensive artifact in the stack to change, because changing it invalidates extractions, prompts, queries, and models downstream. The artifact with the highest cost of churn should not be authored by the component with the least accountability.

Hybrid — LLM proposes, human disposes (recommended, and note it’s the same firewall pattern as extraction, applied one level up): the human owns Steps 1 and 6–8 (competency questions, testing, governance — the parts that encode *your goals*), the LLM accelerates Steps 2–5 (the parts that encode *the data*). Concretely:

1. *You* write the competency questions. Non-delegable: they are your product requirements.
2. *LLM drafts*: give it 20–30 representative documents plus the questions — “Propose the smallest ontology (entity types, relation types with domain/range/symmetry, one-line definitions) sufficient to represent the facts in these documents and answer these questions. Flag facts you chose NOT to model and why.” The output is a fast, coverage-complete first draft that has actually read your corpus.
3. *You curate*: merge near-duplicate types, cut everything not serving a question (expect to cut a third), fix node-vs-property decisions against your ML roadmap, align names with existing standards. This is hours, and it’s where your understanding gets built — you still get manual design’s chief benefit, at a discount.
4. *LLM red-teams*: “Here is my ontology and my questions. Which questions can’t be answered? Which realistic facts in documents like these can’t be expressed? What will extraction find ambiguous?” LLMs are excellent at this adversarial reading — it mechanizes Step 6.
5. *You paper-test, encode, and own it*. From here on, evolution follows Step 8’s governance, with the LLM as a proposer of extensions (fed by the validator’s rejection log) and you as the reviewer, permanently.

So: is there value in doing it manually by studying the data yourself? Yes — and the hybrid workflow is designed to *keep* that value (the understanding, the ownership, the goal-alignment) while delegating what the LLM is genuinely better at (exhaustively reading samples, first-draft coverage, spotting blind spots, consistent naming). What you should never delegate is the decision authority, because the ontology is a set of long-lived commitments about your product, and the LLM doesn’t know your product — it only knows your text.

2. Step 1 — The Ontology in Code (code/ontology.py)

This file is Step 7 of the methodology in Section 1.5 made concrete: the design decisions (from the competency questions, the node-vs-property tests, the granularity choices) frozen into a reviewable, versionable artifact. We express the ontology as plain data so it can serve three masters: the **LLM prompt** (rendered as text to constrain extraction), the **validator** (rejecting non-conformant triples), and **humans** (documentation).

```
@dataclass(frozen=True)
class EntityType:
    name: str
    description: str
    attributes: tuple = ()

@dataclass(frozen=True)
class RelationType:
    name: str
    description: str
```

```

domain: str          # allowed head entity type
range: str           # allowed tail entity type
symmetric: bool = False

```

The professional-world ontology:

ENTITY TYPES:

- Person: An individual professional.
- Company: An employer / organisation.
- Skill: A technology, tool, or competency (e.g. Python, SQL).
- School: A university or other educational institution.
- Location: A city where a person is based or a company operates.

RELATIONSHIP TYPES (head_type -RELATION-> tail_type):

- Person -WORKS_AT-> Company
- Person -PREVIOUSLY_AT-> Company
- Person -HAS_SKILL-> Skill
- Person -STUDIED_AT-> School
- Person -KNOWS-> Person (symmetric)
- Person -BASED_IN-> Location
- Company -USES_TECH-> Skill

The crucial piece is the **validator** — the ontology acting as a gatekeeper:

```

def validate_triple(self, head_type, relation, tail_type):
    rt = self.relation_types.get(relation)
    if rt is None:
        return False, f"unknown relation '{relation}'"
    if rt.domain != head_type:
        return False, f"domain violation: {relation} needs head {rt.domain}"
    ...

```

Running python ontology.py:

```

(True, 'ok')                                # Person WORKS_AT Company
(False, 'domain violation: WORKS_AT needs head Person, got Company')
(False, "unknown relation 'MARRIED_TO'")

```

Design notes worth internalizing: WORKS_AT vs PREVIOUSLY_AT encodes *time* coarsely — production systems instead put start_date/end_date properties on a single EMPLOYED_AT edge (a *temporal* KG). KNOWS is marked symmetric so the graph builder can materialize both directions. And every relation gets a natural-language description — that text is *for the LLM*, and quality here directly improves extraction quality.

3. Step 2 — LLM Ingestion: Text → Triples (code/extract_llm.py)

Why is extraction a separate step at all — why not just hand documents to an LLM at question time? Because reading is expensive and questions are cheap. Extracting once turns $O(\text{corpus})$ work into a reusable structure; thereafter every question costs a graph traversal, not a re-read of everything. It also makes answers *aggregable* (no LLM can hold ten thousand bios in context, but a graph can) and *auditable* (each fact carries its source). Extract once, query forever.

This is also where LLMs replaced a decade of NLP pipelines (NER models + relation classifiers + coreference resolvers). One well-prompted call does all three jobs. The input is ten LinkedIn-style documents (code/sample_data/documents.json), e.g.:

“Amina Yusuf is a Senior Software Engineer at Google in London. She has been building

large-scale recommendation systems since 2019 and is an expert in Python, TensorFlow, and Kubernetes. Amina graduated from Imperial College London... She frequently collaborates with Ben Carter..."

3.1 The extraction prompt — five principles

```
SYSTEM_PROMPT = """You are an information-extraction engine. You convert text
into knowledge-graph triples that STRICTLY conform to a given ontology.
Rules:
- Only use the entity types and relationship types defined in the ontology.
- Only extract facts explicitly stated or unambiguously implied by the text.
- Do NOT invent facts. If unsure, omit the triple.
- Use canonical, full names for entities ("Ben Carter", not "Ben").
- Respond with JSON only, no prose."""
```

The user message contains the ontology (rendered by `to_prompt_block()`), an exact JSON output schema, and the document. The principles:

1. **Closed vocabulary.** The ontology is in the prompt; anything outside it gets rejected afterward anyway. Constrain, then verify.
2. **Structured output.** Demand JSON matching an explicit schema. `temperature=0` — extraction is ETL, not creative writing.
3. **Evidence field.** Every triple must carry a short supporting quote. This (a) suppresses hallucination — inventing facts is harder when a quote is required, (b) gives you instant human review, (c) becomes edge provenance in the graph.
4. **One document per call.** Deterministic, parallelizable, and errors stay contained. Chunk long documents to a few thousand tokens with overlap.
5. **Canonical names.** Ask for full names to reduce the entity-resolution burden downstream.

3.2 Validation: the hallucination firewall

```
data = json.loads(raw)
for t in data["triples"]:
    ok, reason = onto.validate_triple(t["head_type"], t["relation"], t["tail_type"])
    (kept if ok else rejected).append((t, reason))
```

Every triple passes through the ontology validator; violations are logged and dropped. This is the single most important engineering pattern in LLM-powered KG construction: **the LLM proposes, the ontology disposes**. Your graph's integrity never depends on the model being perfect.

3.3 Entity resolution (the hard 20%)

Extraction from two documents yields "Google" and "Google LLC"; "KTH" and "KTH Royal Institute of Technology". If they become separate nodes, the graph silently fragments and every downstream algorithm degrades. The tutorial uses the honest small-scale version — normalization plus an alias table:

```
def canonical(name):
    name = re.sub(r"\s+", " ", name.strip())
    name = re.sub(r"\b(LLC|Inc\.?|Ltd\.)$", "", name).strip(" ")
    return ALIASES.get(name, name)
```

The production ladder, in order of sophistication: (1) normalization + alias tables; (2) **blocking** (group candidates by type + name similarity) then pairwise scoring with string metrics or embedding cosine similarity; (3) an LLM adjudicating ambiguous pairs ("Are 'J. Smith, ML engineer at Google' and 'John Smith, Google Research' the same person? Answer with evidence."); (4)

human review queues for low-confidence merges. LinkedIn spends enormous effort exactly here — mapping free-text job titles and skills onto canonical taxonomy nodes.

3.4 Run it

```
ANTHROPIC_API_KEY=sk-... python extract_llm.py      # → extracted_triples.json
```

A checked-in copy of the result lives at `sample_data/extracted_triples.json` (36 entities, 55 triples, every one with evidence and source doc), so the rest of the tutorial runs with no key. Inspect a sample:

```
{"head": "Ben Carter", "head_type": "Person", "relation": "PREVIOUSLY_AT",  
  "tail": "Meta", "tail_type": "Company",  
  "evidence": "spent four years at Meta", "source_doc": "doc_02"}
```

4. Step 3 — Building the Graph (`code/build_graph.py`)

We load validated triples into a NetworkX MultiDiGraph (directed, parallel edges allowed — a person can have several distinct relations to the same company):

```
G = nx.MultiDiGraph()  
for e in data["entities"]:  
    G.add_node(e["name"], type=e["type"], **e["attributes"])  
for t in data["triples"]:  
    G.add_edge(t["head"], t["tail"], key=t["relation"],  
              relation=t["relation"],  
              evidence=t["evidence"], source_doc=t["source_doc"])
```

Three details that matter: **dedupe** on (head, relation, tail) since the same fact appears in multiple documents; **materialize symmetric relations** (KNOWS gets added in both directions); and **keep provenance on every edge** — evidence and source doc ride along, so any answer the system ever gives can be traced back to a sentence in a source document.

Real output:

```
Nodes: 36   Edges: 60  
  Company (7): Amazon, DeepMind, Google, Grab, Meta, OpenAI ...  
  Person  (8): Amina Yusuf, Ben Carter, Chen Wei, David Okoye, ...  
  Skill   (9): Data Analysis, JAX, Kafka, Kubernetes, PyTorch, Python ...  
  School  (7)   Location (5)  
Relations: {'BASED_IN': 6, 'HAS_SKILL': 16, 'KNOWS': 14, 'PREVIOUSLY_AT': 3,  
            'STUDIED_AT': 8, 'USES_TECH': 5, 'WORKS_AT': 8}  
Saved -> kg.graphml
```

The GraphML file opens in Gephi/yEd for visualization and imports into Neo4j. Speaking of which — the same load in Cypher, for when your graph outgrows RAM:

```
UNWIND $triples AS t  
MERGE (h {name: t.head})    SET h:$(t.head_type)  
MERGE (x {name: t.tail})    SET x:$(t.tail_type)  
MERGE (h)-[r:$(t.relation)]->(x)  
SET r.evidence = t.evidence, r.source_doc = t.source_doc
```

MERGE (get-or-create) is Neo4j's idempotent upsert — it does the dedupe for you. Rule of thumb for choosing a store: NetworkX to $\sim 10^5$ edges and single-process analytics; Neo4j/Memgraph for persistence, concurrent queries, and Cypher; Neptune/TigerGraph/Spark GraphFrames at billions of edges.

5. Step 4 — Querying the Graph (code/query_graph.py)

Now the graph starts paying rent. Each subsection answers one competency question, with the algorithm, the code, the real output, and the Cypher equivalent.

5.1 “Which node is the most busy?” — Centrality

“Busy” or “important” has several precise meanings; choose deliberately:

Degree centrality — raw edge count. Activity.

```
deg = {n: G.degree(n) for n in nodes}
```

```
== Most connected PEOPLE ==  
Amina Yusuf      degree=12  
David Okoye       degree=10  
Elena Petrova     degree=10
```

Betweenness centrality — how many shortest paths pass *through* you. Brokers and bottlenecks: remove them and the network fragments. Note the ranking changes:

```
== Brokers (betweenness) ==  
Priya Nair        0.314    ← bridges the Amazon/Bangalore world to the rest  
Chen Wei          0.277    ← the only link into Meta/Singapore/Grab  
Ben Carter        0.273
```

Priya has *fewer* edges than Amina but more brokerage — she connects communities that otherwise wouldn’t touch. On LinkedIn, high-betweenness members are exactly the “super-connectors” whose introductions unlock new clusters.

PageRank — recursive importance: you matter if important nodes point at you.

```
== PageRank ==  
Kubernetes    0.048      Python    0.046      PyTorch    0.038
```

On this heterogeneous graph PageRank surfaces *skills*, not people — many well-connected people and companies point at them. Same algorithm, different lens.

Cypher equivalent (Neo4j Graph Data Science library):

```
CALL gds.pageRank.stream('kg') YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).name AS name, score ORDER BY score DESC LIMIT 5
```

5.2 “How are X and Y connected?” — Hidden paths

Shortest path in the undirected view of the KG is the warm-introduction chain:

```
== Hidden path: Fatima Al-Sayed -> Priya Nair ==  
Fatima Al-Sayed --HAS_SKILL--> Python  
Python <--HAS_SKILL-- Priya Nair
```

Fatima (junior engineer, Stockholm) and Priya (science lead, Bangalore) share no acquaintances — but they’re two hops apart *through a skill node*. This is why skills belong in the graph as nodes: paths route through them. LinkedIn’s “how you’re connected” and its degree-of-connection badges are exactly this query, restricted to KNOWS edges and computed on a trillion-edge graph. In Cypher:

```
MATCH p = shortestPath((a {name:'Fatima Al-Sayed'})-[*..6]-(b {name:'Priya Nair'}))  
RETURN [n IN nodes(p) | n.name], [r IN relationships(p) | type(r)]
```

5.3 “Find people like this” — Pattern matching

Multi-hop patterns are the bread and butter of KG search (“Python engineers at Google”, “alumni of my school at company X”). In Cypher this is a MATCH pattern; in NetworkX it’s a small comprehension over typed edges:

```
# MATCH (p:Person)-[:HAS_SKILL]->(:Skill {name:'PyTorch'}) RETURN p
== Pattern: people with PyTorch ==
Ben Carter @ Google      Priya Nair @ Amazon      David Okoye @ DeepMind
```

This is *faceted search over structure* — no text matching, no embeddings, just typed edges. It’s also the skeleton of recommendation: “jobs at companies that use technologies you know” is a 3-hop pattern.

5.4 “What groups exist?” — Community detection

Modularity-based clustering (greedy_modularity_communities) finds densely connected regions:

```
C0: Amazon, Bangalore, David Okoye, DeepMind, Priya Nair, PyTorch, Python, ...
C1: Amina Yusuf, Google, Grace Kim, Kubernetes, TensorFlow, ...
C2: Elena Petrova, Fatima Al-Sayed, KTH, Kafka, Scala, Spotify, Stockholm
C3: Chen Wei, Grab, INSEAD, SQL, Singapore
```

The algorithm rediscovered, with zero labels, the real-world structure we planted: a PyTorch-research cluster, a Google-infrastructure cluster, a Spotify/Stockholm cluster, and Chen Wei’s Singapore product world. At scale (use Louvain or Leiden — `python-louvain`, `graspologic`), communities drive feed personalization, cohort analytics, and “groups you may like”.

6. Step 5 — Machine Learning on the Graph: Predicting Missing Links

The questions “what is the next possible link?”, “people you may know”, and “which skills does this person have but hasn’t listed?” are all one problem: **link prediction** — given the graph as it is, score pairs of nodes on how likely an edge is missing or will form. The KG built above is deliberately incomplete in realistic ways (Ben Carter, a Google ML engineer, lists Python and PyTorch but not TensorFlow or Kubernetes — plausible for someone who just never updated his profile). Let’s find those gaps three ways, in increasing order of sophistication. This laddering is the pedagogically important part: each rung explains the next.

6.1 Rung 1 — Neighborhood heuristics (code/link_prediction_heuristics.py)

Classical, interpretable, shockingly strong baselines. For a candidate pair (u, v), look at their shared neighbors:

- **Common Neighbors:** $|N(u) \cap N(v)|$
- **Jaccard:** $|N(u) \cap N(v)| / |N(u) \cup N(v)|$ — overlap normalized by total reach
- **Adamic-Adar:** $\sum_{z \in \text{shared}} 1/\log(\deg(z))$ — shared neighbors weighted by *rarity*. Sharing “Python” (everyone has it) means little; sharing a niche skill or a small startup means a lot. This one insight — rare shared context is informative — powers half of practical recommendation.

Key move: we compute these on the **full heterogeneous graph**, not just the social (KNOWS) layer, so two strangers who share an employer, a school, and three skills still score highly:

```
== People you may know (Adamic-Adar on full KG) ==
Ben Carter <-> David Okoye    AA=1.73    shared: Amina Yusuf, PyTorch, Python
Ben Carter <-> Chen Wei      AA=1.44    shared: Meta
Amina Yusuf <-> Elena Petrova AA=1.24    shared: Ben Carter, Kubernetes
```

Ben Carter <-> Grace Kim AA=1.08 shared: Amina Yusuf, Google

Every suggestion is self-explaining — the shared list is the “you both know Amina” copy LinkedIn shows under a PYMK card.

For **skill inference** we hand-craft *metapaths* — typed multi-hop patterns that constitute evidence:

```
+1.0  Person -WORKS_AT-> Company -USES_TECH-> Skill      (employer's stack)
+0.5  Person -KNOWS-> Person -HAS_SKILL-> Skill          (colleagues' skills)
+0.5  Person -STUDIED_AT-> School <-STUDIED_AT- Person -HAS_SKILL-> Skill
```

== Skills you probably have (metapath scoring) ==

```
Ben Carter -> Kubernetes   score=2.0   because: Google uses it; Amina Yusuf has it; ...
Ben Carter -> TensorFlow    score=1.5   because: Google uses it; Amina Yusuf has it
Grace Kim   -> Python       score=1.5   because: OpenAI uses it; Amina Yusuf has it
Amina Yusuf -> PyTorch      score=1.0   because: Ben Carter has it; David Okoye has it
```

There it is: “**a software engineer at Google likely knows TensorFlow even if they haven’t listed it**” — derived from two metapaths and a weighted count. No neural network required. The limitation is equally clear: we chose the metapaths and the weights. Rung 2 and 3 learn them from data.

6.2 Rung 2 — Knowledge-graph embeddings from scratch (code/embeddings_numpy.py)

The idea underneath all modern graph ML: **learn a vector per node such that the graph’s structure is recoverable from vector arithmetic**. Then “how likely is this missing edge?” becomes “how compatible are these two vectors?” — and compatibility generalizes beyond edges ever observed.

We implement a minimal **DistMult** in pure NumPy (~60 lines, no frameworks) so nothing is magic. Each node gets a vector e , each *relation type* gets a vector w , and a triple is scored:

$$\text{score}(h, r, t) = \sum_k e_h[k] \cdot w_r[k] \cdot e_t[k] \quad P(\text{edge}) = \sigma(\text{score})$$

Training is logistic regression with **negative sampling**: for every true triple, sample corrupted triples (random tail), push $\sigma(\text{score})$ toward 1 for real, 0 for fake, by plain SGD. The gradients are three lines:

```
g = sigmoid(score) - label
E[h] -= LR * g * R[r] * E[t]
R[r] -= LR * g * E[h] * E[t]
E[t] -= LR * g * E[h] * R[r]
```

Real run (400 epochs, dim 32, loss 3.47 → 0.67):

== Predicted missing HAS_SKILL links ==

```
Grace Kim      -> PyTorch      P=0.74   ← her OpenAI team uses PyTorch
Grace Kim      -> Kubernetes   P=0.64
Fatima Al-Sayed -> PyTorch      P=0.17   ← the docs say she's learning it!
```

Nobody told the model about metapaths; it learned that “person embedded near OpenAI-ish, ML-ish nodes” is compatible with “PyTorch” under the HAS_SKILL relation. The Fatima→PyTorch prediction is a lovely detail — the source text says she’s *currently learning* PyTorch, and the geometry picked up the precursor signals. This family (TransE, DistMult, ComplEx, RotatE — see the pykeen library for production implementations) is called **knowledge graph embedding** and is the workhorse of KG completion.

Its weakness: each node’s vector is a free parameter — memorized, not computed. A brand-new node has no vector (the *cold start* problem), and node features (name text, titles, descriptions) are ignored. Enter GNNs.

6.3 Rung 3 — Graph Neural Networks (code/gnn_link_prediction.py)

A **GNN computes** a node's embedding from its features and its neighborhood, instead of memorizing it. One layer of message passing:

```
h_v' = UPDATE( h_v , AGGREGATE({ h_u : u ∈ N(v) }) )
```

Stack L layers and every node's embedding summarizes its L-hop neighborhood. **GraphSAGE** (the variant we use; also the one designed for industrial inductive settings) aggregates by mean and learns the UPDATE weights. Because embeddings are *functions of features and structure*, a node added tomorrow gets an embedding immediately — this **inductive** property is precisely why LinkedIn uses GNNs to complete its member knowledge graph: hundreds of millions of members, constantly changing, most facts missing.

The standard link-prediction architecture is **encoder-decoder**:

```
class SAGEEncoder(torch.nn.Module):          # encoder: 2-layer GraphSAGE
    def __init__(self, in_dim, hid=64, out=32):
        self.conv1 = SAGEConv(in_dim, hid)
        self.conv2 = SAGEConv(hid, out)
    def forward(self, x, edge_index):
        return self.conv2(F.relu(self.conv1(x, edge_index)), edge_index)

def decode(z, pairs):                        # decoder: dot product
    return (z[pairs[0]] * z[pairs[1]]).sum(dim=-1)
```

Node features here are just one-hot type + normalized degree; in production you would use **text embeddings of the node's name/title/description** — that's where the "LLM" and "GNN" worlds fuse, and it's the single highest-leverage upgrade to this script. Training mirrors Rung 2: existing edges are positives, `negative_sampling()` fabricates non-edges as negatives, binary cross-entropy on the dot products, 10% of edges held out for validation AUC. Real output:

```
PyG graph: 36 nodes, 106 directed edges, 6 features/node
epoch 150  loss 1.060  val AUC 0.530
```

```
== GNN-predicted missing HAS_SKILL links ==
Amina Yusuf -> PyTorch      score=0.96
Ben Carter   -> Kubernetes   score=0.89
Grace Kim    -> Kubernetes   score=0.83
```

```
== GNN-predicted missing KNOWS links ==
Amina Yusuf <-> Priya Nair   score=0.95
Ben Carter   <-> David Okoye score=0.94  ← agrees with Adamic-Adar's #1
```

An honest note a tutorial owes you: with 36 nodes, the validation split is ~10 edges and the AUC is noisy — GNNs *need scale* to beat the heuristics of Rung 1 (on this toy graph they roughly tie, and that's the true state of the field: always benchmark against Adamic-Adar before shipping a GNN). What you should take from Rung 3 is the *shape* of the production recipe, which is identical at a billion edges: features → encoder → decoder → negative sampling → AUC.

Scaling-up map: heterogeneous GNNs that keep relation types distinct (R-GCN, HGT — `torch_geometric.nn.HeteroConv`), neighbor-sampled mini-batching (`NeighborLoader`) so you never materialize the full graph, and two-stage serving (embed offline into a vector index, retrieve with approximate nearest neighbors, re-rank online). That last pattern is how PYMK actually serves: **candidate generation via embedding similarity, then a heavyweight ranker with graph features** like common-neighbor counts — Rung 1 and Rung 3 working together.

6.4 What LinkedIn actually does (case study)

LinkedIn’s engineering blog describes completing their *member knowledge graph* with GNNs, and it maps exactly onto what you just built. Their graph is heterogeneous (members, skills, titles, companies, schools, geos) like ours. Missing profile facts (unlisted skills, implicit seniority, industry) are link-prediction targets, i.e. our Section 6.3 with HAS_SKILL edges. Node features come from text encoders over profile text (our “highest-leverage upgrade”). Training is self-supervised on existing edges with negative sampling — the same recipe as our 60-line NumPy trainer. And the outputs feed everything downstream: better PYMK, better job matching (“qualified for X because the graph infers skills beyond the listed ones”), better search recall, ads targeting, and feed ranking. The inferred edges go *back into the KG* with confidence scores — the graph heals itself. The same loop runs elsewhere: Google’s KG completes facts about entities for search; Amazon’s product graph infers product attributes and complements; Pinterest’s PinSAGE recommends pins — all the encoder-decoder recipe at scale.

7. Step 6 — The Natural-Language Interface (code/nl_query.py)

The final layer: users ask questions in English; answers come from the graph. The pattern is **LLM as router, graph as oracle** — the inverse of Step 2, and governed by one rule: *the LLM never answers from its own knowledge*. It translates questions into calls against the graph functions you built in Steps 4–6, reads the results, and composes a grounded answer.

Mechanically, every graph function becomes a Claude **tool** (name + description + JSON schema):

```
{"name": "most_connected",
  "description": "Nodes with the highest degree (busiest nodes). Optionally
                  filter by node type: Person, Company, Skill, School, Location.",
  "input_schema": {...}},
{"name": "connection_path", ...},
{"name": "predict_connections", ...},    # ← link prediction as a tool!
{"name": "predict_skills", ...},
```

with a system prompt enforcing grounding (“Answer **ONLY** from tool results. Never invent facts about these people. Cite the relation/evidence behind each claim.”). The agentic loop is ~20 lines: send question + tools; while the model requests tool calls, execute them and return results (including *errors* — the model reads them and self-corrects, e.g. retrying “Fatima” as “Fatima Al-Sayed”); when it stops requesting tools, its text is the answer. A session looks like:

```
you> which person is the most busy, and are they a broker too?
[graph call] most_connected({"node_type": "Person"})
[graph call] brokers({})
Amina Yusuf is the most connected person (degree 12), but the biggest
*broker* is Priya Nair (betweenness 0.31) – she bridges the Amazon/
Bangalore cluster to the rest of the network...
```

```
you> what skills does Ben Carter probably have but hasn't listed?
[graph call] predict_skills({})
Kubernetes (score 2.0 – Google uses it, and his collaborators Amina and
Elena both have it) and TensorFlow (score 1.5 – Google standardised on it
and Amina, whom he works with, is an expert)...
```

Notice what the second answer is: a *prediction*, with *evidence*, in *English* — the whole tutorial stack (ontology → extraction → graph → link prediction → NL layer) in one exchange.

Design notes. **Tool-calling vs text2cypher**: the alternative interface is having the LLM write Cypher directly against Neo4j — more expressive (arbitrary queries), but you must guard against invalid or expensive queries; curated tools are safer and self-documenting, so start with tools,

add text2cypher behind a validator when users outgrow them. **GraphRAG**: this architecture is retrieval-augmented generation where the retrieval corpus is a *graph* instead of text chunks — multi-hop questions (“skills of people who work with Amina”) that defeat vector-similarity RAG become single traversals; Microsoft’s GraphRAG and Neo4j’s GenAI stack are productionized versions. **Provenance pays off**: because Step 3 kept evidence/source_doc on edges, the interface can cite the sentence behind every claim — try getting that from a raw LLM.

8. Putting It Together: What Runs on a Knowledge Graph

Step back and see the full taxonomy of workloads a finished KG supports — this is the payoff that justifies the ontology and the pipeline. Five layers, in increasing sophistication: **(1) lookup and multi-hop query** — retrieve an entity’s facts, match typed patterns, follow paths (Sections 5.2–5.3; the workhorse layer, powering search, facets, and profile pages); **(2) graph analytics** — whole-graph algorithms like centrality, communities, connected components (Sections 5.1, 5.4; powering influencer detection, cohorts, fraud rings); **(3) rule-based inference** — deterministic derivation of new edges from declared semantics: symmetry, transitivity, inverses, subclass hierarchies (Section 1.3, return #4; cheap, exact, explainable); **(4) statistical inference** — embeddings, link prediction, GNNs that *guess* missing structure with confidence scores (Section 6; powering recommendations and profile completion); **(5) language interfaces and agents** — LLMs translating human intent into layers 1–4 and composing grounded answers (Section 7). The layers compound: inferred edges from layers 3–4 flow back into the graph that layers 1–2 query and layer 5 narrates.

Every “intelligent” feature on a professional network is a short recipe over these primitives:

Feature	Recipe
Entity search & facets	pattern matching (5.3) + text index on node names
“How you’re connected”	shortest path (5.2)
Influencers / super-connectors	degree, PageRank, betweenness (5.1)
People You May Know	link prediction on KNOWS (6.1/6.3) + shared-evidence explanation
Skill inference / profile completion	link prediction on HAS_SKILL (6.1–6.3)
Job ↔ candidate matching	metapaths Person→Skill←Job, boosted by <i>inferred</i> skills
“Groups you may like” / cohorts	community detection (5.4)
Warm-intro routing (“hidden paths”)	paths through Company/School/Skill nodes (5.2)
Q&A assistant	LLM router over all of the above (7)

Two composition effects are worth savoring. First, **inference compounds**: once skill inference writes Ben →HAS_SKILL→ TensorFlow back into the graph (flagged *inferred*: true, with confidence), every other recipe — job matching, search recall, PYMK — improves for free. Second, **explanations fall out of structure**: recommendations arrive with their reasons (shared neighbors, metapath evidence), which is a UX and trust advantage vector-only recommenders can’t match.

8.1 Production checklist

What separates the tutorial from a production system, in the order it will bite you: **entity resolution at scale** (blocking + embedding similarity + LLM adjudication + human review — this is where the bodies are buried); **incremental ingestion** (documents arrive continuously; extraction is per-document so it parallelizes trivially; use MERGE-style upserts and re-run analytics on

a schedule); **temporal modeling** (start/end dates on edges; facts change); **evaluation** (hold out known edges and measure extraction precision/recall against a hand-labeled sample, plus AUC / hits@k for link prediction — never ship on vibes); **confidence and trust tiers** (extracted facts with evidence > inferred facts with model scores; render them differently); **storage migration** (NetworkX → Neo4j when you need persistence/concurrency — your GraphML file imports directly); and **cost control** (extraction is one LLM call per document *once*, querying is a handful of cheap tool calls — the expensive model is only ever the translator, never the database).

9. Exercises

The gap between reading and mastery is doing. In rough order:

1. **Extend the ontology** with Job postings (Company -POSTS-> Job, Job -REQUIRES-> Skill) and write the 3-hop query “jobs whose requirements this person meets”. Then re-run skill inference and watch matches improve.
2. **Break the extractor**: feed it a bio with a fact your ontology can’t represent (“Amina invested in a startup”) and watch the validator reject it. Then extend the ontology to capture it.
3. **Better entity resolution**: replace the alias table with embedding cosine similarity over entity names (sentence-transformers), then add an LLM adjudication call for pairs scoring 0.7-0.9.
4. **Evaluate properly**: hide 20% of HAS_SKILL edges, then measure hits@3 for Rung 1 vs Rung 2 vs Rung 3 at recovering them. On this tiny graph, expect the heuristics to win — understand why.
5. **Text features**: replace the one-hot GNN features with embeddings of each node’s name and type sentence. This usually lifts the GNN past the heuristics — feel the fusion of LLMs and GNNs first-hand.
6. **Port to Neo4j**: import kg.graphml, rewrite Section 5’s queries in Cypher, and swap nl_query.py’s tools for a text2cypher tool with a query validator.
7. **Scale test**: generate 10,000 synthetic profiles with an LLM, ingest, and find which of your components groans first.

10. Reading list

Foundations: *Knowledge Graphs* (Hogan et al., 2021 survey — the field’s canonical map); Stanford CS224W lectures (graph ML, free online). Graph ML: *Graph Representation Learning* (Hamilton — short, excellent book; free draft online); the GraphSAGE, R-GCN, and PinSAGE papers. Industry: LinkedIn’s engineering blog post “Completing a member knowledge graph with Graph Neural Networks”; Google’s “Introducing the Knowledge Graph” post; Microsoft GraphRAG paper/repo. Tools to grow into: pykeen (KG embeddings), PyTorch Geometric docs (heterogeneous graphs, NeighborLoader), Neo4j Graph Data Science manual, rdflib/Protégé if you need formal OWL ontologies.

Appendix: File map & how to run

```
kg-tutorial/  
├── TUTORIAL.md           ← you are here  
├── code/  
│   ├── ontology.py      Step 1: ontology + validator (no deps)  
│   ├── sample_data/documents.json  10 raw input documents  
│   ├── extract_llm.py    Step 2: Claude extraction (needs API key)  
│   ├── sample_data/extracted_triples.json  pre-extracted result (no key needed)  
│   └── build_graph.py     Step 3: triples → NetworkX → kg.graphml
```

— query_graph.py	Step 4: centrality, paths, patterns, communities
— link_prediction_heuristics.py	Step 5a: Adamic-Adar, metapath skill inference
— embeddings_numpy.py	Step 5b: DistMult from scratch (NumPy only)
— gnn_link_prediction.py	Step 5c: GraphSAGE link prediction (PyG)
— nl_query.py	Step 6: natural-language interface (needs API key)


```

pip install networkx scipy numpy          # steps 1, 3, 4, 5a, 5b
pip install torch torch_geometric        # step 5c
pip install anthropic                    # steps 2 and 6
cd code
python ontology.py && python build_graph.py && python query_graph.py
python link_prediction_heuristics.py && python embeddings_numpy.py
python gnn_link_prediction.py
ANTHROPIC_API_KEY=sk-... python extract_llm.py    # regenerate triples yourself
ANTHROPIC_API_KEY=sk-... python nl_query.py       # chat with your graph

```